AKKA

Reliable AI for Every Industry
The Agentic Systems Platform

Akka vs. LangChain

A comparison for teams building agentic AI

July 2026

LangChain is the fastest way to a prototype and the slowest way to production — the moment you ship, you own availability, durability, governance, and operations. Akka owns all of it under a contractual SLA, and Akka Specify can deliver and run the entire governed system for you as a guaranteed outcome.

99.9999%	Weeks	Fixed	90%
AKKA AVAILABILITY SLA	AKKA SPECIFY DELIVERY	ONE ALL-IN PRICE	LESS INFRASTRUCTURE

DIMENSION	LANGCHAIN	AKKA
What it is	A developer framework and observability product; the customer builds, runs, and operates the application	**A full-stack agentic systems platform — build on it yourself, or have it delivered and operated for you**
How you reach production	Self-build, or contract a labor-led integration engagement	**Build with spec-driven development, or Akka Specify delivers and runs it**
Commercial model	Metered parts (per trace / run / compute) plus build-and-operate labor	**One fixed price — platform, infrastructure, tokens, training, delivery, and operations**
Outcome guarantee	Effort only; the outcome is not guaranteed	**The delivered outcome is guaranteed**
Who owns & operates it	The customer's team	**Akka — a contractual 99.9999% SLA, 24/7 SRE, sub-1 min RTO, zero byte RPO**
State and memory	Checkpointed to an external database on every step — tens of ms, up to ~150ms as state grows	**Embedded in-memory state, active-active replicated: 4ms reads / sub-10ms writes**
Governance / EU AI Act	LangSmith provides tracing and evals; inline enforcement, tamper-evident logging, and sign-offs are the customer's responsibility	**Guardrails, policy enforcement, and hash-chained evidence logging woven into the runtime**
Pre-production governance	Not provided	**Classify against 186 regulations / 877 controls, multi-persona sign-offs, Governance Posture Packages**
Production runtime	Durable runtime with a limited production track record (LangGraph, stable late 2025); customer-operated	**An 18-year-proven runtime across 100,000+ deployments**
Infrastructure footprint	More compute for the same transaction volume, plus separately provisioned state, streaming, vector, and observability services	**Up to 90% less infrastructure for the same agentic transaction volume — actor concurrency + shared compute**
Improves after go-live	LangSmith detects and alerts on quality; improvement is manual	**Continuous evaluation, reinforcement learning, and distillation — up to 80% lower token cost over time**
Portability	LangSmith hosting plus a LangChain-specific state layer	**Any cloud, on-prem, or sovereign deployment; portable specs; BSL licensing**
Real-time streaming	Bolt-on (Kafka / Flink / Kinesis), separately operated	**Built into the runtime — backpressured, petabyte-scale, in-memory, no external broker**

1. Framework vs. Platform: Two Different Problems

Agentic systems run in one of two operating models. **Person-attached** agents run inside a human's session: they inherit the user's identity and stop when the session closes. **Process-attached** agents run unattended, on their own service identity, triggered by events, schedules, or other systems; they must survive crashes, restarts, and infrastructure failures without losing their place. Production agentic AI is overwhelmingly process-attached, and that operating model requires a durable execution substrate. Akka and LangChain's LangGraph both target it, and they meet the requirement differently.

What LangGraph provides

LangGraph persists graph state through **checkpointers** that write to an external store — in-memory, SQLite, or Postgres. A graph can resume after an interruption, pause for human review, and carry memory across runs. LangGraph Platform (offered as LangSmith Deployment) adds a task queue, managed persistence, and durable background runs.

What LangGraph does not provide

- **Embedded vs. external state.** LangGraph checkpoints graph state to an external database on every step — each step pays a serialize-and-round-trip cost of tens of ms, up to ~150ms as state grows, compounding across a multi-step agent. Akka holds durable state embedded in memory, active-active replicated, at 4ms reads / sub-10ms writes — durability is a runtime property, not a database round trip.
- **Infrastructure-grade resilience.** No published active-active HA/DR, sub-1-min RTO with zero-byte RPO, or numeric availability SLA. Recovery is a replay from the last checkpoint, and that store's HA/DR is the customer's responsibility.
- **A vendor-owned availability guarantee.** Akka owns a contractual 99.9999% SLA on the running system; with LangChain, application availability is owned by the customer.

Stability and operational risk in the durable-execution layer

The checkpointer and durable-execution layer that LangGraph applications depend on has open, maintainer-confirmed defects affecting scaling, availability, and performance — all bug-labeled on GitHub and open as of June 2026:

- **Availability — memory leak in the default mode.** With the default `durability="async"`, checkpointer coroutine chains accumulate and leak memory in a long-running process (#7094).
- **Availability — state loss.** Cancelling a run drops streamed state not yet written as a checkpoint (#5672).
- **Availability — persistence failures.** The Postgres checkpointer throws connection/SSL errors recurring across multiple versions (#3716).
- **Performance & scaling.** Checkpoint serialization produces ~85% storage bloat and ~37.8% token overhead with no opt-out (#7714).

These defects affect scaling, availability, and performance, and a team that adopts the stack owns resolving them in its own deployment. In Akka, durable state is embedded in the runtime, replicated active-active, and operated under a contractual SLA; these failure modes are properties of an external checkpoint store, not the Akka runtime.

2. Two Ways to Production: Build It, or Have It Delivered

Akka offers two ways onto one platform: build the system yourself with spec-driven development, or have **Akka Specify** deliver and operate the entire governed system for you as a guaranteed outcome — in weeks, for one fixed price. LangChain offers neither a platform nor a delivery model: a LangChain application is the customer's to build, run, and own.

	With LangChain	With Akka Specify
Model	Self-build, or a labor-led integration engagement	You provide the specifications
Who does the work	Your engineers, or forward-deployed / staff-augmentation contractors	Akka generates, governs, delivers, and runs the system
Timeline	3–12 months, by construction	Weeks, not quarters
Billing	Time-and-materials, plus metered platform parts	One fixed price
Afterward	You inherit the system and its operations	Kept up, safe, and improving — operated by Akka
Guarantee	Effort is guaranteed; the outcome is not	The delivered outcome is guaranteed

By LangChain's own Shared Responsibility Model, application availability, recovery, and operations are the customer's. A team that cannot self-build contracts a labor-led integration engagement — billed for time and materials, guaranteeing effort rather than the outcome, and handing back a system the team then owns and operates. Akka Specify inverts that: you provide a handful of plain-language specifications, and Akka generates, tests, governs, deploys, and runs the system — then keeps it available, safe, and improving under one agreement.

3. Akka Delivers the System; LangChain Delivers Parts

LangChain's portfolio spans the open-source `langchain` framework and LangGraph for durable agent orchestration, the Deep Agents harness, and LangSmith — the commercial agent-engineering platform providing observability, evaluation, and LangSmith Deployment for hosting. The customer assembles and operates these into a running application. Akka ships one pre-integrated platform, and Akka Specify can deliver and operate the finished system.

Dimension	LangChain	Akka
Integrated runtime	Components the customer wires together; the application SLA is owned by the customer's team	One pre-integrated runtime; Akka owns a 99.9999% SLA, sub-1 min RTO, and 24/7 SRE with active-active replication
Delivery & operation	The customer builds and operates the application, or contracts a labor-led integrator	Delivered and operated by Akka Specify as a guaranteed outcome, in weeks
Pricing	Per-trace billing (LangSmith) plus separately provisioned state, streaming, observability, and governance, plus build-and-operate labor	One fixed price on shared compute — platform, infrastructure, tokens, training, delivery, and operations
Governance	Observability and evals via LangSmith; runtime enforcement and conformance built per project	Guardrails, sanitizers, and evidence logging woven into the runtime; conformance proven by Akka Verify

A production LangChain application requires the customer to build and operate a state-persistence layer, failover and recovery logic, an observability pipeline, governance and compliance tooling, and scaling and deployment automation. Akka provides each of these as part of the platform — and, through Akka Specify, will build, deliver, and run the whole system for you.

4. Maturity & the Runtime

LangChain's open-source libraries reached v1.0 in October 2025, with a commitment to no breaking changes until 2.0 — a move toward API stability after the frequent restructurings of the 0.x line. The durable runtime production agents depend on has a limited production track record: LangGraph reached its first stable release only in late 2025, and managed hosting (LangGraph Platform / LangSmith Deployment) is newer still, with BYOC limited to AWS. In every model, the customer assembles the framework, runtime, and hosting into an application and operates it.

Akka's proven runtime

18 years across 100,000+ deployments — 52 banks, systems 2 billion people use daily. Built on actor concurrency (200M actors/core), durable sharded in-memory state (4ms reads, 10ms writes, replayable from its event journal), and brokerless backpressured streaming (1-minute RTO, zero-byte RPO). Akka operates this substrate under a contractual SLA.

5. Availability & DR

LangChain publishes a 99.5% API uptime SLA — for both its Cloud SaaS and its BYOC control plane — measured quarterly with service credits. It does not publish an availability SLA for the customer's application or agents, HA/DR for application state, or automatic failover — application availability is owned by the customer. 99.5% permits ~43.8 hours of downtime per year; Akka's 99.9999% permits ~31 seconds, and Akka owns *and operates* it for the entire running system, 24/7.

Metric	LangChain	Akka
Published SLA	99.5% API uptime (SaaS & BYOC)	99.9999% whole platform
App / agent availability SLA	Not published	Guaranteed
Who owns availability	The customer	Akka
Who operates it	The customer (or an inherited integrator)	Akka SREs, 24/7
HA mode	Customer-built	Active-active
RTO / RPO	Customer-implemented	<1 min / zero byte
State during failover	Customer's responsibility	Fully preserved
SLA backing	Service credits (control plane)	Contractual indemnities

6. Governance and the EU AI Act

The penalties are enforceable now

Violation	Maximum Fine
Prohibited AI practices (Art. 5)	EUR 35M or 7% global turnover
High-risk obligations (Art. 9-15)	EUR 15M or 3% global turnover
Incorrect information to authorities	EUR 7.5M or 1.5% global turnover

High-risk AI carries a 10-year logging-retention obligation under Article 72. Akka indexes 186 AI regulations and 877 controls (288 carrying financial penalties) and derives the applicable obligation set automatically.

What LangSmith provides

LangSmith — LangChain's observability and evaluation product — is positioned for EU AI Act readiness: end-to-end **tracing** for action logging (Art. 12), execution-graph **transparency** (Art. 13), **evaluators** for bias, toxicity, hallucination, PII leakage, and prompt injection (Art. 10, 15), **human oversight** via LangGraph's `interrupt` primitive and annotation queues (Art. 14), client-side PII masking of trace data, EU/BYOC/self-hosted residency, and SOC 2 Type II.

These capabilities **observe and evaluate**; the developer wires enforcement into the graph. By LangChain's own account, the policy work and risk-management-system design are the customer's responsibility, and LangSmith does not provide tamper-evident logging, pre-deployment risk classification, sign-off workflows, or a sealed audit artifact.

EU AI Act, Side by Side

Requirement	LangSmith (LangChain)	Akka
Action logging (Art. 12)	End-to-end tracing of calls, tools, and steps; 14- or 400-day retention	Hash-chained, runtime-witnessed records, retained for the Article 72 horizon
Record integrity	Trace records in LangSmith's store; tamper-evidence not claimed	Tamper-evident, hash-chained

Transparency (Art. 13)	Execution-graph visualization and step inspection	**Self-explanation as a runtime property of every decision**
Risk controls and evaluation (Art. 10, 15)	Evaluators for bias, toxicity, hallucination, PII leakage, prompt injection — detect and alert	**Inline guardrails, policies, LLMs-as-a-judge, and sanitizers**
Enforcement point	**Observes and evaluates; blocking is wired into the graph by the developer**	**Enforced inline in the runtime, at the agent boundary**
Human oversight (Art. 14)	LangGraph `interrupt` (coded into the graph) plus annotation-queue review	**Runtime control plane: pause, override, or redirect any running process**
PII handling	Client-side masking of trace data	**Decision, PII scrub, and explanation produced atomically at runtime**
Data residency	EU SaaS, BYOC, self-hosted	**Any cloud, on-prem, or sovereign deployment**
Pre-deployment risk classification	**Not provided**	**186 regulations / 877 controls, classified before a system ships**
Multi-persona sign-off workflows	**Not provided**	**Declarative recipe engine with dossiers and quorum**
Sealed audit artifact	**Not provided**	**Governance Posture Package, ready for regulatory handoff**

7. Cheaper to Operate — and It Keeps Getting Cheaper

AI systems built with Akka are up to **90% cheaper to operate than Python-based systems**. The difference is how much infrastructure each approach needs to handle the same volume of agentic transactions: a Python-based stack spreads the work across the application plus separately provisioned services — a database for state, a streaming or message tier, a vector store, and an observability backend, each sized with its own headroom. Akka runs orchestration, agents, memory, streaming, and APIs on one shared-compute runtime and carries the same transaction load on far fewer cores. Three runtime properties produce the gap:

- **Actor-based concurrency** packs far more concurrent work onto each core — ~10 trillion tokens per core per year versus ~2 trillion for comparable solutions, and ~80% less compute than Python-based frameworks. After porting its Python-based systems to Akka, Manulife reported up to 300% more concurrency and 30–50% faster processing.
- **Shared compute** runs APIs, agents, memory, orchestration, and streaming on one runtime rather than separately provisioned services, eliminating the duplicated infrastructure and idle headroom each standalone service carries.
- **Micro-checkpointing** of durable actions resumes work from the last completed step instead of re-running it, minimizing retries — cutting wasted compute and the repeated LLM calls that drive token cost.

And the cost falls after go-live

The delivered outcome compounds. Akka Verify runs continuous evaluation, reinforcement learning on production and synthetic data, and distillation to smaller specialized models — cutting token cost **up to 80%** while raising accuracy over time. LangSmith detects and alerts on quality; it does not retrain, distill, or ship the improvement. With Akka Specify, that tuning is part of the operated outcome, not a project the customer runs later.

The spend is also predictable: **one fixed price finance can forecast** — covering platform, infrastructure, tokens, training, delivery, and operations — rather than consumption metering (per trace, per run, per compute unit) that moves with load, plus separate build-and-operate labor.

8. Portability

A production LangChain application accumulates dependencies on LangSmith for observability (commercial, per-trace), a state layer built specifically for LangChain's patterns, and separately integrated governance tooling. Migrating off LangChain means replacing each of these. LangGraph Platform offers a managed BYOC deployment, but it is currently AWS-only.

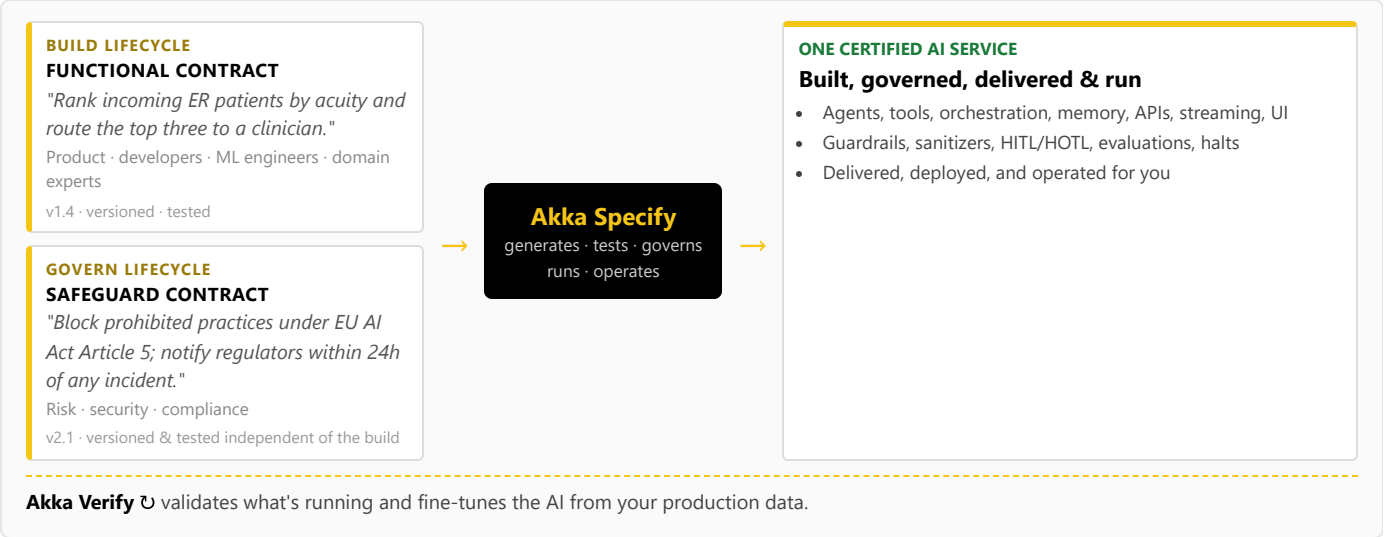
Akka deploys on:

- Akka's cloud, the customer's hyperscaler VPC, the customer's own Kubernetes, or on-premises
- A sovereign cloud option with country-isolated data, networking, and operations
- Portable specs — the spec is the source of truth and enables agent portability across environments
- BSL licensing

9. One Certified System — Built, Governed, Delivered, and Run

LangChain is a code-first developer framework: building a LangChain application requires software engineers writing Python or TypeScript, and any safeguards are written by those same engineers inside the application code — there is no independent place for risk, security, and

compliance to own governance, and no path for a product manager or domain expert to contribute. Akka runs **two independent lifecycles** on one platform, and **Akka Specify** lets each audience work in plain language at its own level of abstraction — a **build lifecycle** (the Dev Spec, authored, versioned, and tested by product, developers, ML engineers, and domain experts) and a **governance lifecycle** (the Eval Matrix, defined, versioned, and tested by risk, security, and compliance *independently of the AI system itself*):



Akka generates, tests, governs, **runs, and operates** one certified AI service that satisfies both specifications, and **Akka Verify** validates the running system against both. Governance is an independent, versioned, testable lifecycle owned by the people accountable for it — not a developer afterthought in application code — and, through Akka Specify, the whole certified system is delivered and operated as a guaranteed outcome. LangChain has no equivalent for either the independent governance lifecycle or the delivery model.

10. Real-Time Streaming at Petabyte Scale

LangChain has no native stream-processing engine; real-time data pipelines are provisioned and operated separately (Kafka, Flink, Kinesis, or equivalent) and bolted onto the application. Akka's streaming is built into the runtime: continuous, backpressured event flows across components, services, and regions, with no external broker. It processes **petabyte-scale data in memory** with end-to-end backpressure, so fast producers never overwhelm slow consumers and no events are dropped under load. This powers not only agent feedback loops but high-throughput real-time data processing — the engine behind Tubi's real-time hyper-personalization at 5 billion tokens per second. It is a class of real-time, high-volume workload LangChain does not address.

11. For the Buyer: Risk, Compliance, and Accountability

Buyer concern	LangChain	Akka
Vendor track record	Company founded 2023 (Series B)	Profitable; 18 years and 100,000+ production deployments (52 banks); Dell Technologies Capital is largest shareholder, a customer, and an AI partner
Certifications & audits	SOC 2 Type II (LangSmith)	19 standards — SOC 2 Type II + public SOC 3, ISO/IEC 27001 & 42001, HIPAA, PCI DSS, GDPR, NIS2, DORA, EU AI Act, NIST AI RMF — plus annual penetration tests, SBOMs, and 40+ documented security policies (trust.akka.io)
Delivery & outcome	Self-build or a time-and-materials integrator; effort is guaranteed, the outcome is not	Akka Specify delivers and operates the system for one fixed price; the outcome is guaranteed
Risk transfer	Availability, recovery, and EU AI Act liability sit with the customer	Availability and data-integrity guarantees backed by contractual indemnities
Accountability	You are the integrator and the on-call	Akka owns the SLA and runs 24/7 SRE — one accountable vendor
Budget predictability	Consumption-metered (per trace, per run, per compute unit), plus build-and-operate labor	One fixed price finance can forecast
Adoption	—	Incremental; Akka can run alongside existing LangChain/LangGraph work

For an enterprise buyer, the security questionnaire, the liability model, and vendor longevity gate the deal before any feature comparison — and that is where the gap is widest.

Customers Running Agentic and Real-Time Systems on Akka

Manulife
2,000
developers across 100 projects on one governed platform

Tubi
5B tok/s
real-time hyper-personalization engine

Swiggy
71ms
order-assignment AI, ~50% latency reduction

John Deere
1,000+
tractor sensors turned into real-time insight

Verizon
750%
order-processing capacity gain; 6s → 2.4s response

Common Questions

We don't have a team to build this — what are our options?

Two. Build it on the platform with spec-driven development, or have Akka Specify deliver and operate it for you. You provide plain-language specifications; Akka generates, governs, delivers, and runs the system as a guaranteed outcome, for one fixed price. With LangChain, production is self-build or a labor-led integration engagement you then own.

How is Akka Specify different from hiring an integrator to build our LangChain app?

An integration engagement sells effort — time-and-materials over months — and hands back a system you own and operate; the outcome is not guaranteed. Akka Specify sells the outcome: a governed system delivered in weeks for one fixed price, then kept up, safe, and improving by Akka. You own the specifications, not the operational burden.

Does our LangChain experience transfer to Akka?

Yes. Agent patterns, prompt engineering, and LLM integration skills apply directly. Akka adds the production layer LangChain does not provide — durable state, HA/DR, governance, and operational guarantees — and spec-driven development lets a team describe the system in plain language and generate it in hours, or Akka Specify can deliver and operate it for you.

We use LangGraph for orchestration. What does Akka add?

Akka provides the same stateful orchestration plus the substrate underneath it: durable in-memory state replicated across the cluster, active-active HA/DR with zero-byte RPO, a vendor-owned availability SLA, and governance woven into the runtime. LangGraph orchestrates; Akka orchestrates and guarantees the running system.

LangGraph Platform deploys our agents. Isn't that production?

LangGraph Platform (LangSmith Deployment) serves and hosts applications and publishes a 99.5% control-plane uptime SLA under BYOC, currently AWS-only. It does not publish an application-level availability SLA, active-active HA/DR with zero-byte RPO, embedded governance, or pre-deployment classification. Akka owns a 99.9999% SLA on the running system, on any cloud.

LangChain is open source. Does Akka create lock-in?

The `langchain` framework is open source. A production LangChain deployment also depends on LangSmith for observability, a LangChain-specific state layer, and separately integrated governance. Akka runs on any cloud or on-premises, keeps the spec portable across environments, and ships under BSL licensing.

Is a platform like Akka heavier than moving fast on a framework?

Time to a prototype and time to production are different measures. Production requirements — HA/DR, governance, compliance, and observability at scale — are where framework-based projects slow down. Akka's spec-driven development ships systems with HA and governance already in place — described in plain language, then generated, tested, and run by the platform — or delivered and operated for you by Akka Specify.

Sources

LangChain & LangSmith pricing: langchain.com/pricing — Developer $0 / 5k traces; Plus $39/seat / 10k; $2.50/1k base, $5.00/1k extended; deployment $0.005/run, $0.0036/min, $0.05/fleet run, $1.50/LCU

LangGraph & LangChain v1.0: langchain.com/blog/langchain-langgraph-1dot0 — first stable release, no breaking changes until 2.0

LangGraph Platform deployment & SLA: langchain.com/support-plans — Self-Hosted Lite, Cloud SaaS, BYOC (AWS-only), Self-Hosted Enterprise; 99.5% control-plane uptime

LangGraph persistence / checkpointers: langchain-ai.github.io/langgraph — graph state checkpointed to an external store on each step

LangGraph durable-execution defects (open, June 2026): github.com/langchain-ai/langgraph/issues — #7094 default-mode memory leak, #5672 state loss on cancellation, #3716 Postgres checkpointer failures, #7714 checkpoint serialization bloat

LangSmith Shared Responsibility Model: docs.langchain.com/langsmith/shared-responsibility-model — customer owns application-level HA/DR

LangChain on the EU AI Act: langchain.com/blog/langsmith-langchain-oss-eu-ai-act — tracing/logging, evaluators, LangGraph interrupt, residency

LangSmith trace data masking: docs.langchain.com/langsmith/mask-inputs-outputs — client-side PII masking of trace data

Akka Specify (spec-driven delivery): akka.io / akka.io/llms.txt — you provide the specifications; Akka generates, tests, governs, delivers, and operates the system as a guaranteed outcome; one fixed price covering platform, infrastructure, tokens, training, delivery, and operations; delivered in weeks; continuous improvement via reinforcement learning and distillation (up to 80% lower token cost, higher accuracy)

Akka trust center: trust.akka.io — 19 compliance standards; SOC 2 Type II + public SOC 3; ISO 27001/42001, HIPAA, PCI DSS, GDPR, NIS2, DORA, EU AI Act, NIST AI RMF; annual pen tests, SBOMs, 40+ policies

Akka performance & efficiency: akka.io/blog/go-slow-to-go-fast — Manulife up to 300% more concurrency, 30–50% faster processing after porting Python; ~10T tokens/core/year vs ~2T, ~80% less compute than Python frameworks

Akka platform: 99.9999% availability, active-active HA/DR, sub-1 min RTO, zero byte RPO (contractual indemnities); 186 regulations / 877 controls / 288 with financial penalties; 100,000+ deployments over 18 years